



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 07 – Lecture 24: Course Revision and Consolidation

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Seven units, one argument.

Topic focus: **which method, and how would you know you were wrong**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, O'Reilly, 2016.; Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.; Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012.; Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Morgan Kaufmann, 2012.

The spine
oooo

One dataset
oooooooo

Choosing
oooo

Five mistakes
oooooooooooo

End to end
ooooo

The arithmetic
oooooo

Revision map
oo

Assessment
ooo

Summary
oo

Quick Links

The spine

One dataset, every model

Choosing

Five mistakes

End to end

The arithmetic that recurs

Revision map

What is left to be assessed

Summary

Agenda

Seven units, one argument

One dataset, every model

Which method, and why

The five mistakes that cost the most

From raw arrays to a defended result

The arithmetic that recurs across units

The revision map

What is left to be assessed

Summary

How this lecture works

This is not a list of topics

You already have twenty-three sets of notes that list topics. The job today is to turn them into **one argument**, so that when you are handed a problem you have never seen you can say which method to try, why, and what would tell you it was the wrong choice.

Commit before you look

Five slides ask you to choose a method, or to find the error in a block of code, *before* the answer appears. These are diagnostic. Getting one wrong now is cheap; getting it wrong in an examination is not.

Everything quoted today was **re-measured for this lecture** under one protocol — one dataset, one pipeline, twenty folds — rather than copied from the earlier packages.

One seed for the whole revision, and two sweeps

The randomness

One seed: `random_state=24` and `np.random.default_rng(24)`. Every model, every split, every shuffle, every synthetic set. The outer loop of a nested CV uses 25 so that it is not the inner loop.

Two sweeps

Mistakes 1 and 2 are measured on **twenty fresh noise problems** each, from two different streams. One trial of a noise experiment proves nothing.

The one thing a seed cannot fix

Every number in this deck reproduces exactly *except the milliseconds*. Quote what is countable — 3201 weights against 31 coefficients, 100 trees, 106 support vectors — never the clock.

The four questions the whole course answers

1. **What quantity are we minimising?**
Unit II
2. **How is the minimisation carried out?**
Unit III
3. **Over what class of functions?**
Units V, VI
4. **How would I know it worked?**
Units IV and VI

And before all four: **what do you feed it?** — Unit IV again, because the columns you supply decide what the optimiser can possibly find.

Unit VII is the same picture with a hole in it

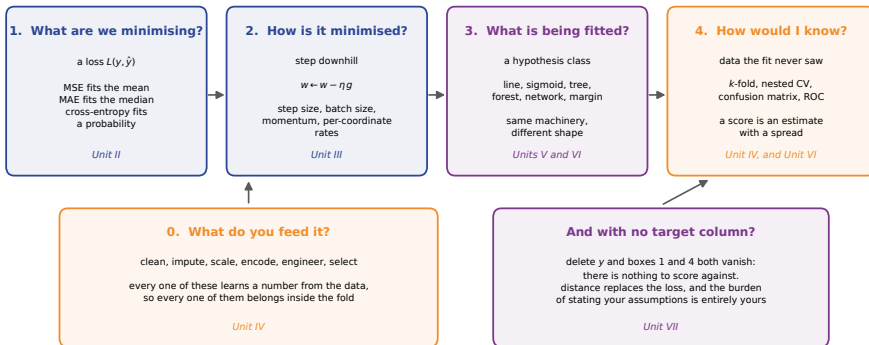
Delete the target column and questions 1 and 4 both lose their meaning. There is no loss to minimise and nothing to score against. What replaces the loss is a *distance*, and what replaces the score is your own stated assumption.

The sentence to carry out of the room

Least squares, logistic regression, trees, forests, boosting, networks and SVMs are **not seven inventions**. They are one

The whole course on one page

One argument, not seven units: choose a loss, minimise it over a class of functions, on columns you chose, and then prove it on data the fit never saw



Same recipe, seven fillings

Method	Hypothesis class	Loss	Minimised by
Least squares	linear in x	squared error	closed form
Ridge / lasso	linear in x	squared + penalty	closed form / coordinate descent
Logistic	linear in log-odds	cross-entropy	gradient descent, IRLS
Decision tree	axis-aligned boxes	impurity drop	greedy search
Random forest	average of trees	impurity drop	greedy + bootstrap
Boosting	sum of trees	any differentiable	stagewise gradient
Neural network	composed layers	any differentiable	backpropagation + SGD
SVM	linear in a feature map	hinge + $\ w\ ^2$	convex QP
k -NN	the training table	— (no fit)	—

Read the table *down the columns*, not across the rows. Every row is the same three decisions.

What each unit actually established

Unit II — the loss is a choice

Six readings, one corrupted: the MSE-minimising constant moves from 12.33 to 25.67, past every clean observation. The MAE-minimising constant does not move at all.

Unit III — descent is three questions

How big a step, from how much data, with how much memory. At $\eta = 1.00$ the iterates cycle $0 \rightarrow 6 \rightarrow 0$; at $\eta = 1.05$ they reach -17.18 in twenty steps.

Unit IV — the input decides the ceiling

Select features with the labels before cross-validating and 500 pure-noise columns score 0.8475; inside a Pipeline, 0.5342.

Units V and VI — classes and losses

Same optimiser, different shapes. Unit VII removes the target and makes the distance the whole decision: k -means on raw wine scores ARI 0.3711, standardised 0.8975.

The protocol, written once and used for everything

```
X, y = load_breast_cancer(return_X_y=True) # 569 x 30, 357 benign  
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=4, random_state=24)
```

```
for name, est in seven_models.items():  
    pipe = Pipeline([("sc", StandardScaler()), ("m", est)])  
    # fit on 4/5, score on 1/5, twenty times; the scaler is refitted  
    each time
```

Twenty folds, not five, so that the fold-to-fold spread is measurable rather than guessed.
The scaler is *inside* the pipeline, which is the whole point of Unit IV.

The question this experiment exists to settle

When two models differ by a few thousandths, is that a difference?

Predict first #1

Commit to an answer

Seven models — logistic regression, k -NN, a single tree, a random forest, gradient boosting, an MLP and an RBF SVM — on 569 patients and 30 columns, in one honest cross-validated pipeline.

1. Which wins on **accuracy**?
2. Which wins on **AUC**?
3. How large is the gap between the top two, and how large is the fold-to-fold standard deviation?

Predict first #1

Commit to an answer

Seven models — logistic regression, k -NN, a single tree, a random forest, gradient boosting, an MLP and an RBF SVM — on 569 patients and 30 columns, in one honest cross-validated pipeline.

1. Which wins on **accuracy**?
2. Which wins on **AUC**?
3. How large is the gap between the top two, and how large is the fold-to-fold standard deviation?

Reveal

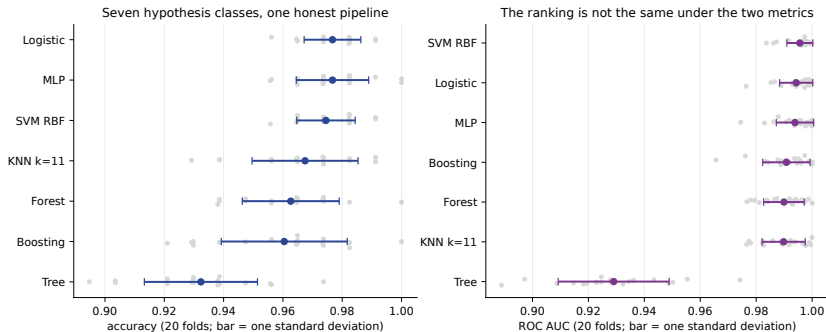
Accuracy: **logistic regression, 0.9767**. AUC: **the RBF SVM, 0.9956**. The AUC gap to second place is **0.0013**; the fold-to-fold standard deviation is **0.0046**. The gap is about a quarter of the noise. **Four of the seven** models sit within one fold-standard-deviation of the best accuracy.

The measured table

model	acc	sd	AUC	sd	fit ms	pred ms
LogisticRegression	0.9767	0.0096	0.9943	0.0059	2.3	0.24
KNN (k=11)	0.9675	0.0179	0.9898	0.0077	0.6	1.39
DecisionTree	0.9324	0.0191	0.9290	0.0198	6.1	0.35
RandomForest	0.9626	0.0163	0.9899	0.0073	341.3	7.08
GradientBoosting	0.9605	0.0212	0.9908	0.0085	366.9	0.62
MLP (100,)	0.9767	0.0122	0.9939	0.0067	225.9	0.44
SVM (RBF)	0.9745	0.0099	0.9956	0.0046	2.5	0.71

Six of the seven are separated by less than 0.017 of accuracy. The seventh — the single unpruned tree — is the only one that is clearly worse, and it is also the only one with no averaging or regularization anywhere in it. The two millisecond columns are a measurement on one machine; the accuracies are not.

Drawn: the seven, with the fold cloud behind them



The grey dots are the individual folds. The bars overlap almost completely.

Predict first #2 — is 0.0013 a difference?

Commit

The SVM's mean AUC beats logistic regression's by 0.0013, over twenty paired folds. A paired t -test gives $p = 0.0232$. *Would you report the SVM as the better model?*

Predict first #2 — is 0.0013 a difference?

Commit

The SVM's mean AUC beats logistic regression's by 0.0013, over twenty paired folds. A paired t -test gives $p = 0.0232$. *Would you report the SVM as the better model?*

Reveal

The test is significant and the difference is **immaterial**. The nominal winner **lost 6 of the 20 folds**. On accuracy the same two models differ by +0.0022 in the *other* direction, with a paired t -test $p = 0.3965$: logistic wins 7 folds, loses 4 and **ties 9**.

Predict first #2 — is 0.0013 a difference?

Commit

The SVM's mean AUC beats logistic regression's by 0.0013, over twenty paired folds. A paired t -test gives $p = 0.0232$. *Would you report the SVM as the better model?*

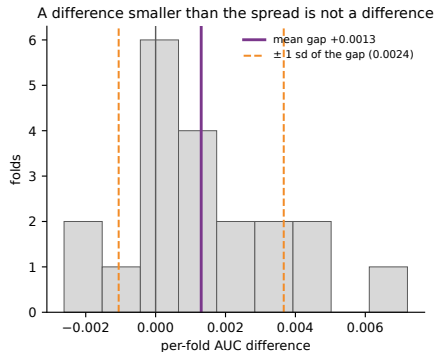
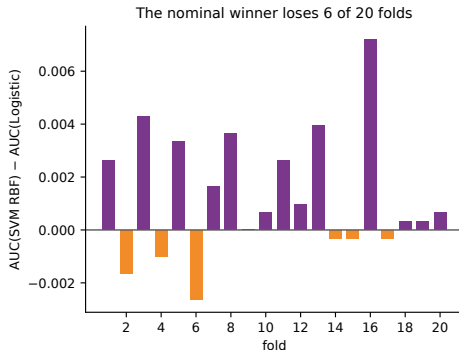
Reveal

The test is significant and the difference is **immaterial**. The nominal winner **lost 6 of the 20 folds**. On accuracy the same two models differ by +0.0022 in the *other* direction, with a paired t -test $p = 0.3965$: logistic wins 7 folds, loses 4 and **ties 9**.

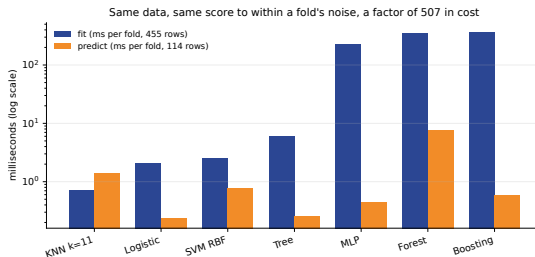
The rule

A difference smaller than the fold-to-fold spread is not a difference. Report the spread whenever you report the score, and prefer the cheaper or the more interpretable model when the intervals overlap.

Drawn: the gap against the spread



And what the seven cost



100 trees and 1406 nodes of gradient boosting buy nothing measurable over **31 coefficients** of logistic regression. *k*-NN is the mirror image: nothing fitted, dearest to predict. (Counts, not seconds — the seconds are machine-dependent.)

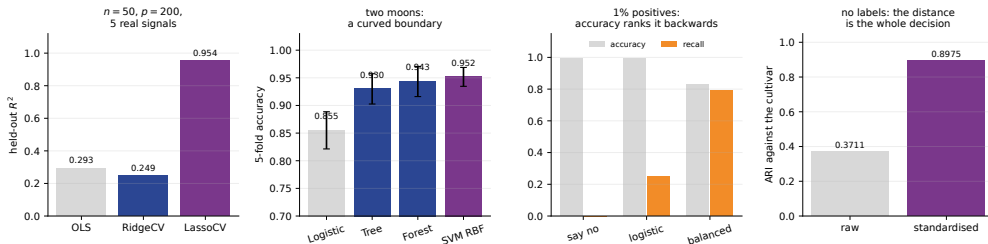
The ranking depends on the metric

rank	by accuracy	by AUC
1	LogisticRegression	SVM (RBF)
2	MLP (100,)	LogisticRegression
3	SVM (RBF)	MLP (100,)
4	KNN (k=11)	GradientBoosting
5	RandomForest	RandomForest
6	GradientBoosting	KNN (k=11)
7	DecisionTree	DecisionTree

models whose rank changes with the metric : 5 of 7

Five of seven models change rank when you change the metric. There is no metric-free notion of “the best model” — so name the metric before you name the winner, and choose the metric from the problem, not from the leaderboard.

Choosing is not folklore — every row here is a number



Four regimes in which the seven models are *not* interchangeable. In each one the choice is worth far more than any hyper-parameter sweep.

The choosing table

Situation	Try first	Because, measured
Few rows, many columns, sparse truth	lasso, then elastic net	$n = 50, p = 200$, five real signals: OLS $R^2 = +0.2926$, ridge $+0.2492$, lasso $+0.9543$ and it recovered 5 of the 5 real columns
Interpretability required	penalised logistic or a shallow tree	on <code>breast_cancer</code> , 30 readable coefficients score AUC 0.9947 against gradient boosting's 0.9919: the readable model cost nothing
Non-convex boundary	RBF SVM, k -NN, or a forest	two moons: logistic 0.8550, tree 0.9300, forest 0.9433, SVM 0.9517
Heavy imbalance	any model, but change the <i>metric</i> and the <i>threshold</i>	at 1% positives, always-say-no scores accuracy 0.9900; the model that finds 19 of 24 positives scores 0.8275
No labels at all	clustering, and spend the effort on the dis-	k-means on wine: raw ARI 0.3711, standardised 0.8975 — one column carried 0.77% of

Predict first #3 — four described problems

Commit to one method for each, and to one reason

1. 80 patients, 12,000 gene-expression columns, and you must tell the clinician *which* genes.
2. 2 million rows, 15 columns, a prediction must return in under a millisecond.
3. 3,000 transactions, 30 of them fraudulent, and a missed fraud costs a hundred times a false alarm.
4. 50,000 customer records with no label of any kind, and the marketing team wants “segments” .

Predict first #3 — four described problems

Commit to one method for each, and to one reason

1. 80 patients, 12,000 gene-expression columns, and you must tell the clinician *which* genes.
2. 2 million rows, 15 columns, a prediction must return in under a millisecond.
3. 3,000 transactions, 30 of them fraudulent, and a missed fraud costs a hundred times a false alarm.
4. 50,000 customer records with no label of any kind, and the marketing team wants “segments”.

Reveal

1 lasso or elastic net — it is the only family that returns a *list*; ridge never returns a zero, its smallest coefficient on a swept path was 1.455×10^{-3} . **2** logistic regression — not k -NN, whose predict cost grows with n (1.11 ms $\rightarrow$ 25.56 ms for $40\times$ the rows). **3** any model, plus average precision, plus a threshold from the 10:1 cost — *not* accuracy. **4** k -means or DBSCAN, but first standardise and be able to defend the distance; and say out loud that there is no answer key.

Two answers that are always wrong

“I tried all of them and picked the best score”

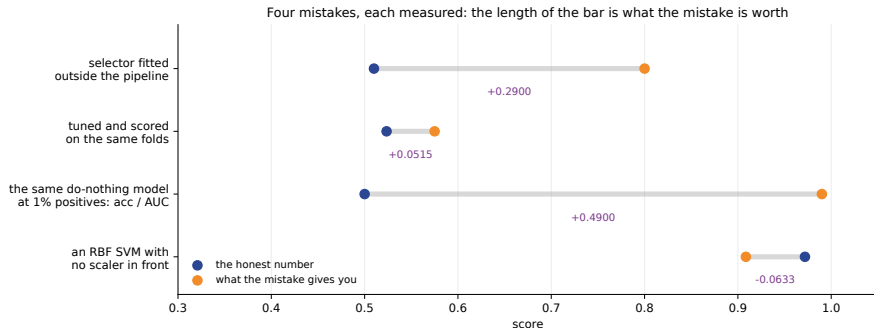
That is a hyper-parameter search with the test set as the objective. On 100 rows of pure noise a 22-way model search reports **0.5750** against a nested 0.5235, when the truth is 0.5000 — and it overstates on **17 of 20** trials.

“Deep learning, because it is the most powerful”

On these 569 patients the MLP scores 0.9767 and logistic regression scores 0.9767 — identical to four decimals — and the MLP costs **103×** as many parameters (3201 against 31). Capacity you do not need is capacity you have to regularise, tune and defend.

The defensible answer names a property of the *problem* — $p \gg n$, curved boundary, prevalence, cost asymmetry, the need to explain — and then names the family that property implies.

Four of the five, measured



The fifth is not a score at all: it is reading a coefficient as a cause.

Mistake 1 — a transformer fitted outside the pipeline

```
X = rng.normal(size=(100, 500)) # 500 columns of pure noise
y = rng.integers(0, 2, size=100) # coin-flip labels: truth is 0.5

sel = SelectKBest(f_classif, k=10).fit(X, y) # <-- sees every row
wrong = cross_val_score(LogisticRegression(), sel.transform(X), y, cv=5)

right = cross_val_score(Pipeline([("k", SelectKBest(f_classif, k=10)),
                                   ("m", LogisticRegression())]), X, y, cv=5)
```

```
selection outside the pipeline    0.8000
selection inside the pipeline     0.5100
overstatement                     +0.2900
over 20 fresh noise problems: leaky 0.7535, honest 0.5000
the leaky protocol scored higher on 20 of 20
```

Why that happens, in one sentence

The mechanism

SelectKBest looked at the labels of *every* row, including the rows that later became the validation fold. Ten of five hundred coin-flip columns will always look predictive by chance — and once they have been chosen using the answers, the validation fold is no longer unseen.

The rule that covers all of Unit IV in one line

Anything that learns a number from the data — a mean, a median, a category list, a rotation, a feature subset, a resampled row, a hyper-parameter — is part of the model and belongs inside the fold.

The costs measured across the course: leaking a scaler +0.0034; leaking an imputer, nothing measurable; leaking a row-dropping decision +0.1702 of R^2 ; resampling before the split, $F_1 = 0.8942$ against an honest 0.1133.

Mistake 2 — tuning and evaluating on the same folds

```
gs = GridSearchCV(pipe, grid_of_22_models, cv=inner).fit(X, y)
print(gs.best_score_) # <-- this is a MAXIMUM
```

```
nested = cross_val_score(GridSearchCV(pipe, grid_of_22_models, cv=inner),
                          X, y, cv=outer) # <-- this is an ESTIMATE
```

```
breast_cancer    best_score_ 0.9824    nested 0.9824 +/- 0.0125
                  optimism      -0.0000
100x30 pure noise, coin-flip labels, 20 trials; truth 0.5000
  best_score_ 0.5750    nested 0.5235    optimism +0.0515
  best_score_ exceeded the nested estimate on 17 of 20
```

On an easy, well-separated problem the optimism is **nothing**; on a hard one it is the whole apparent signal. You cannot tell which case you are in *without* the nested loop — and that is the argument for always running it.

Mistake 3 — accuracy under imbalance

test prevalence 0.0100, 24 positives in 2400 rows

model	acc	AUC	AP	recall
always say no	0.9900	0.5000	0.0100	0.0000
logistic, default	0.9917	0.9084	0.4690	0.2500
logistic, class_weight	0.8275	0.8938	0.3035	0.7917

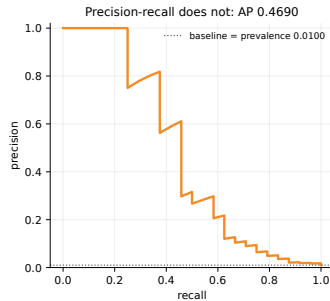
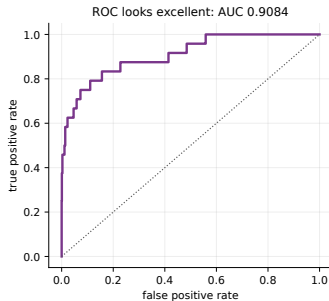
Read the last row

The only model that *finds* the positives — 19 of the 24 — is the one accuracy ranks **last**, behind a model that has never predicted a positive in its life.

What to quote instead

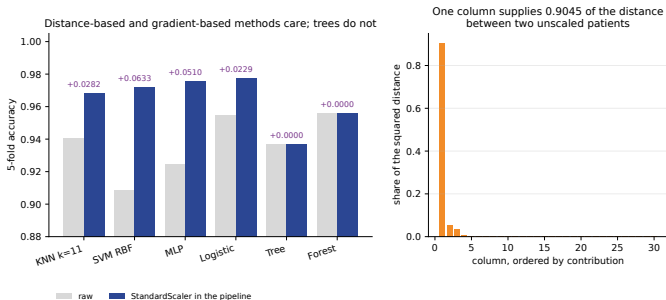
Recall and precision at a stated threshold, average precision (baseline = prevalence, here 0.0100), and the confusion matrix itself. **Never accuracy alone.**

ROC flatters where precision–recall does not



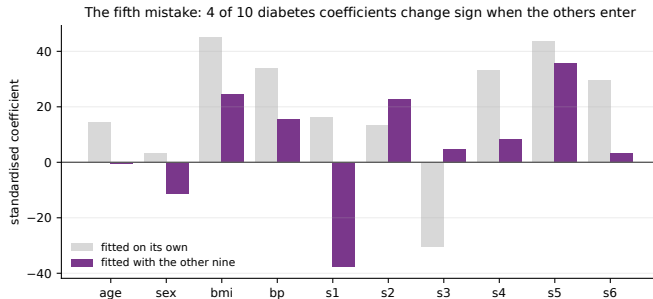
Same model, same scores, same test rows. ROC AUC is invariant to the class balance; average precision is not, and at 1% positives that is the honest one.

Mistake 4 — no scaler in front of a distance or a gradien



The tree and the forest are **exactly** unchanged (+0.0000), because a split on $x_j < t$ is invariant to any monotone rescaling of x_j . Everything that measures a distance or takes a gradient is not.

Mistake 5 — reading a coefficient as a cause



Total serum cholesterol has slope $+0.4723$ on its own and -1.0900 with the other nine predictors present. Both are correct. They answer *different questions*.

The confounder, built on purpose so the truth is known

The design

$Z \sim \mathcal{N}(0, 1)$; $X = Z + \varepsilon_1$; $Y = 3Z + \varepsilon_2$.

X appears **nowhere** in the equation for Y . Its causal effect on Y is exactly zero, by construction.

What the regressions say

Y on X alone: slope **+2.3662**, $R^2 = 0.7790$.

Y on X given Z : slope **-0.0119**.

True causal effect: **+0.0000**.

The wording that is defensible

“Holding the other predictors in this model fixed, a one-unit increase in x_j is *associated with* a change of b_j in the fitted mean of y .” Not “causes”. Not “if we increase x_j ”. And never without naming which other predictors were in the model.

Predict first #4 — find the errors

```
X = StandardScaler().fit_transform(X)
X = SelectKBest(f_classif, k=20).fit_transform(X, y)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2)
gs = GridSearchCV(SVC(), {"C": [0.1, 1, 10]}, cv=5).fit(Xtr, ytr)
print("test accuracy", gs.score(Xte, yte), "best CV", gs.best_score_)
```

Predict first #4 — find the errors

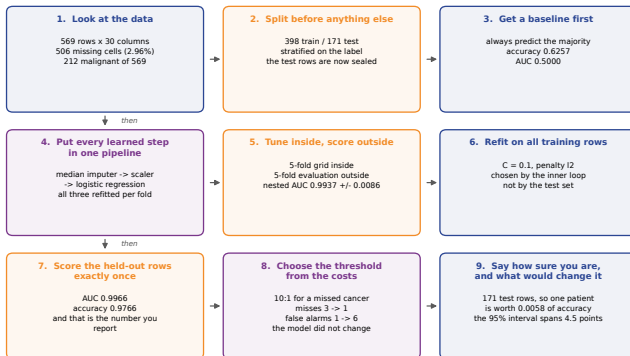
```
X = StandardScaler().fit_transform(X)
X = SelectKBest(f_classif, k=20).fit_transform(X, y)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2)
gs = GridSearchCV(SVC(), {"C": [0.1, 1, 10]}, cv=5).fit(Xtr, ytr)
print("test accuracy", gs.score(Xte, yte), "best CV", gs.best_score_)
```

Reveal — there are four

(1) the scaler is fitted on all rows, before the split; (2) the selector is fitted on all rows *and all labels* — the +0.2900 mistake; (3) `train_test_split` is not stratified and has no `random_state`; (4) `best_score_` is printed beside the test score as though the two were comparable estimates. Also: one accuracy on 20% of the rows, with no spread.

Every step of the course, in the order you would really do

Raw arrays to a defened result: the order is the content



Steps 1–3: split, baseline, one pipeline

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.30,  
                                     stratify=y, random_state=24)  
base = DummyClassifier(strategy="most_frequent").fit(Xtr, ytr)  
  
pipe = Pipeline([("imp", SimpleImputer(strategy="median")),  
                 ("sc", StandardScaler()),  
                 ("m", LogisticRegression(max_iter=5000,  
                                           solver="liblinear"))])
```

```
step 0  569 x 30, 506 missing cells (2.96%), 229 complete rows of 569  
        positive class = malignant, 212 of 569 rows (0.3726)  
step 1  train 398, test 171, prevalence 0.3719 train / 0.3743 test  
step 2  majority class: accuracy 0.6257, AUC 0.5000
```

Listwise deletion would have thrown away **340 of 569** patients to remove 2.96% of the cells.
That is the exponential $(1 - p)^d$ from Unit IV, in one line.

Steps 4–6: tune inside, score outside, score once

```
grid = {"m__C": [0.001, 0.01, 0.1, 1, 10, 100], "m__penalty": ["l1", "l2"]}
nested = cross_val_score(GridSearchCV(pipe, grid, cv=inner,
                                       scoring="roc_auc"),
                          Xtr, ytr, cv=outer, scoring="roc_auc")
gs = GridSearchCV(pipe, grid, cv=inner, scoring="roc_auc").fit(Xtr, ytr)
```

```
step 4 nested CV on the training rows: AUC 0.9937 +/- 0.0086
step 5 best params {'m__C': 0.1, 'm__penalty': 'l2'}, best_score_ 0.9941
      best_score_ - nested = +0.0004
step 6 held-out rows, scored ONCE: AUC 0.9966, accuracy 0.9766
      confusion matrix  TN 106  FP 1  FN 3  TP 61
      recall 0.9531    precision 0.9839
```

The test set was touched exactly once, at step 6, and never again. If you look at it and then change something, it is a validation set and you no longer have a test set.

Steps 7–8: the threshold is a separate decision

threshold	TN	FP	FN	recall	precision
0.5000 default	106	1	3	0.9531	0.9839
0.4778 Youden	106	1	3	0.9531	0.9839
0.2525 cost	101	6	1	0.9844	0.9130

step 8 171 test rows: one patient is worth 0.0058 of accuracy;
binomial se 0.0116, 95% interval [0.9540, 0.9993]

The number that matters clinically

Moving the threshold from 0.5 to 0.2525 takes missed malignancies from **3 to 1** and false alarms from 1 to 6. *Nothing about the model changed* — only the price list.

Reported honestly

Youden's J , chosen on the out-of-fold scores, picked 0.4778 and **changed nothing at all** on the test rows. That is what happened; it is not a failure of the method, it is what an equal-cost rule does on a well-segregated problem.

Step 9 and 10 — what a defended result actually says

The sentence to write in a report

“A median-imputed, standardised, L_2 -penalised logistic regression ($C = 0.1$, chosen by a 5-fold inner loop) scored a nested cross-validated AUC of **0.9937** $\pm$ **0.0086** on 398 training patients and **0.9966** on 171 held-out patients, against a majority-class baseline of accuracy 0.6257 and AUC 0.5000. At a threshold chosen for a 10:1 miss-to-false-alarm cost it missed **1 of 64** malignancies at the price of 6 false alarms. The test set has 171 rows, so one patient is worth 0.0058 of accuracy.”

And then the honest caveat

“This is one cohort from one institution. A prevalence shift, a different imaging protocol, or a column that encodes the outcome would all invalidate it, and none of those is tested here.”

Least squares, and the ridge that generalises it

Five points, all integers

$x = (1, 2, 3, 4, 5)$, $y = (2, 4, 5, 4, 5)$.

$\bar{x} = 3$, $\bar{y} = 4$, $S_{xx} = 10$, $S_{xy} = 6$.

$$b_1 = \frac{S_{xy}}{S_{xx}} = 0.6, \quad b_0 = \bar{y} - b_1\bar{x} = 2.2$$

$SSE = 2.40$, $S_{yy} = 6.00$, $R^2 = 0.6000$,
 $\sum e_i = 0$ and $\sum x_i e_i = 0$.

Ridge, no intercept, $x = (1, 2, 2)$,
 $y = (2, 3, 5)$

$$\hat{\beta}(\lambda) = \frac{x^T y}{x^T x + \lambda} = \frac{18}{9 + \lambda}$$

λ	0	1	9	81
$\hat{\beta}$	2.0	1.8	1.0	0.2

Ridge reproduces all four to four decimals.
 $\lambda \rightarrow \infty$ shrinks towards zero but never reaches
 it.

Entropy and information gain — fourteen rows

$H(S) = H(9/14, 5/14) = 0.940286$ bits

Sunny (2 yes, 3 no) weight $5/14$ $H = 0.970951$

Overcast (4 yes, 0 no) weight $4/14$ $H = 0.000000$

Rain (3 yes, 2 no) weight $5/14$ $H = 0.970951$

Remainder(Outlook) = 0.693536 Gain(Outlook) = 0.246750

Gain(Humidity) = 0.151836 Gini(S) = 0.459184

Outlook wins because it is the only attribute with a **pure child**: four Overcast days, all Yes, entropy exactly 0. That single zero-entropy branch is worth $4/14 \times 0.940286 = 0.2686$ of the remainder on its own.

The one to remember

$H(p) = - \sum p_i \log_2 p_i$; gain = parent entropy minus the *weighted* average of the children's. The weights are what stop a fourteen-way identifier split from being free.

One backward pass, nine parameters

$x = (1, 0.5)$, target $t = 1$, sigmoid everywhere, $L = \frac{1}{2}(a_2 - t)^2$.

$$W^{(1)} = \begin{pmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{pmatrix},$$

$$b^{(1)} = (0.1, 0.1),$$

$$W^{(2)} = (0.5, -0.5), b^{(2)} = 0.2.$$

$$\delta_2 = (a_2 - t) a_2(1 - a_2);$$

$$\delta_1 = \delta_2 W^{(2)} \odot a_1 \odot (1 - a_1).$$

```
z1 = [0.300000, 0.600000]  a1 = [0.574443, 0.645656]
z2 = 0.1643931053  a2 = 0.5410059687  L =
0.1053377604
delta2 = -0.1139767142
dL/dW2 = [-0.06547307, -0.07358978]
dL/db2 = -0.11397671
delta1 = [-0.01393128, 0.01303804]
max |analytic - central difference| = 2.095e-11
```

One forward pass and one backward pass give *all nine* derivatives. Nine separate finite differences would need eighteen forward passes — and that ratio is what makes networks trainable at all.

Bayes, and one k -means iteration

Prevalence 1%, sensitivity 99%,
specificity 95%

$$P(D | +) = \frac{0.99 \times 0.01}{0.99 \times 0.01 + 0.05 \times 0.99} = \frac{0.0099}{0.0599}$$

In 10,000 people: **99** true positives arrive
alongside **495** false ones. Odds form:
 $0.010101 \times 19.8 = 0.2$.

Six points, $k = 2$, one Lloyd step

Centres (1, 1) and (5, 7). Assignment
 $\{P_1, P_2, P_3\}$ and $\{P_4, P_5, P_6\}$ — note P_3 is
equidistant at 3.6056 and the tie goes to
the lower index. New centres
(1.8333, 2.3333) and (4.3333, 5.6667); W
falls from 24.7500 to **10.6667**.

Both halves of a Lloyd iteration can only decrease W , which is why k -means always terminates
— and why it terminates at a *local* minimum.

A confusion matrix, and one point on a ROC curve

TP 6	FP 2	FN 4	TN 8	(n = 20)
accuracy	0.7000			
precision	0.7500			
recall	0.6000			
specificity	0.8000			
FPR	0.2000			
F1	0.6667			
ROC point	(0.2000, 0.6000)			
Youden J	0.4000			

Ten scored rows → the whole curve

Sort by score, step *up* for a positive and *right* for a negative. AUC = **0.8400** by trapezoids, and **21/25** concordant pairs — the same number, because AUC *is* the probability that a random positive outscores a random negative.

The trap

`confusion_matrix` is the **transpose** of the textbook layout: top-left is TN, bottom-right is TP.

Predict first #5 — three claims, one is defensible

Which of these would you sign your name to?

1. "Our model achieves 99.2% accuracy on fraud detection."
2. "Random forest beat logistic regression, 0.9626 against 0.9767 — wait, the other way round — so we chose the forest."
3. "Nested cross-validated AUC 0.9937 ± 0.0086 ; on 171 held-out patients, 0.9966; baseline 0.5000."

Predict first #5 — three claims, one is defensible

Which of these would you sign your name to?

1. "Our model achieves 99.2% accuracy on fraud detection."
2. "Random forest beat logistic regression, 0.9626 against 0.9767 — wait, the other way round — so we chose the forest."
3. "Nested cross-validated AUC 0.9937 ± 0.0086 ; on 171 held-out patients, 0.9966; baseline 0.5000."

Reveal

Only **3**. Claim **1** quotes accuracy on an imbalanced problem — at 1% positives, always-say-no scores 0.9900 — with no baseline, no recall and no spread. Claim **2** picks a winner from a gap of 0.0141 without ever measuring the fold-to-fold spread, which here is 0.0163: *larger than the gap*. Claim **3** names the protocol, the spread, the held-out number and the baseline.

Unit by unit: the one idea, and what is examinable

Unit	The one idea	The calculations that are genuinely examinable
I	Learning is fitting a function from examples; three kinds of task	naming the task type; MAE / MSE / RMSE on a handful of numbers
II	Choosing a loss chooses what the model becomes	minimising a loss over a constant; MSE → mean, MAE → median; writing down cross-entropy
III	Step downhill: how big, from how much data, with how much memory	one gradient-descent step by hand; the divergence condition; one momentum and one Adam update
IV	What you feed it decides what it can find	$(1 - p)^d$ row survival; the z, IQR and modified-z rules; a standardisation by hand; what belongs inside the fold
V	Least squares has an answer you can write down, and λ controls it	$b_1 = S_{xy}/S_{xx}$; R^2 ; the normal equations on a 2×2 ; ridge $\hat{\beta} = X^T(X + \lambda I)^{-1}Y$

What to reread, and in what order

If you have one evening

The four boxes of the spine figure, and then the six by-hand calculations in Section 6: least squares, information gain, one backward pass, Bayes, one Lloyd iteration, and a confusion matrix with its ROC point. Those six recur across the whole paper.

If you have a week

Add the derivations: why least squares is the Gaussian MLE, why $X^T X + \lambda I$ is always invertible, why the logistic log-likelihood is concave, why a bootstrap sample holds 63.2% of the unique rows, and why single linkage chains.

What is *not* worth memorising

Library argument names. The exact coefficient table of Lance–Williams. Any number you can recompute in ten seconds. Spend the time on the arithmetic you can be asked to *produce* rather than recognise.

The remaining internal components

What is still to come

- **Theory quizzes.** T10 is the last of the ten, covering Unit VII's algorithms — hierarchical, partitioning and density-based clustering. 40 minutes, like every other one.
- **Lab assignments and lab quizzes.** LA10 covers experiments 13 and 14; LQ10 follows it. Each lab quiz is 60 minutes at the machine, on *your own* submitted code.
- **Project.** Final submission — code, report and reproducibility notes — and then the **individual project quiz**.

Dates

Every remaining date is announced on the LMS and in class, at least one session ahead. Do not infer one from the syllabus.

The project quiz, since it is the largest single piece left

How it is structured

Of the 15 project marks, **10 are individual** — a written paper on your own group's project, one set per theme, with every member sitting it separately — and **5 are for the group**, identical for every member, awarded for the state *and the commit history* of the submitted repository.

What it asks for

Your project, in your words: what problem, what data, what you cleaned and why, what models you compared, on what protocol, with what spread, and what you would do differently. Every one of those is a question this course has an answer to.

The one preparation that helps

Be able to run your own pipeline end to end and explain each line of it. Two thirds of the project mark is individual, so a strong contributor in a weak group can still score well — and the reverse is also true.

What good work looks like from here

In the report

A baseline. A pipeline with every learned step inside it. A cross-validated score *with its spread*. One held-out number, scored once. A metric chosen from the problem. A threshold chosen from the costs. And a paragraph on what would change your mind.

In the paper

Why that model and not another — naming a property of the data. Why that metric. What the spread was. What you tried that did not work, reported as it happened. A null result you can explain is worth more than a good number you cannot.

Reproducibility is a mark, not a courtesy: a fixed seed, a pinned version list, and a script that runs from a clean checkout.

What to carry out of this course

The argument

1. Choose a loss — it decides what the model becomes.
2. Minimise it — the how does not depend on the what.
3. Over a class of functions — the class is the modelling decision, and it is a statement about the shape you believe the answer has.
4. On columns you chose — and everything you learn from the data belongs inside the fold.
5. Prove it on data the fit never saw —

The five numbers

+0.2900 — a selector outside the pipeline.
+0.0515 — tuning and scoring on the same folds.
0.9900 — the accuracy of doing nothing at 1% positives.
+0.0633 — an RBF SVM with a scaler in front of it.
+0.4723 → -1.0900 — a coefficient that is not a cause.

And the honest one

On this dataset, seven very different models are separated by less than the fold-to-fold



Where to go next

Immediately

The notes for this lecture carry a **revision map** for all seven units, fifteen practice questions weighted towards the recurring hand calculations, and full worked solutions.

Afterwards

Convolutional and recurrent networks, probabilistic graphical models, reinforcement learning, and the whole of deployment — monitoring, drift, fairness. Everything in this course is the vocabulary those subjects assume.

It has been a pleasure. Go and measure something.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, O'Reilly, 2016.; Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.; Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012.; Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Morgan Kaufmann, 2012.